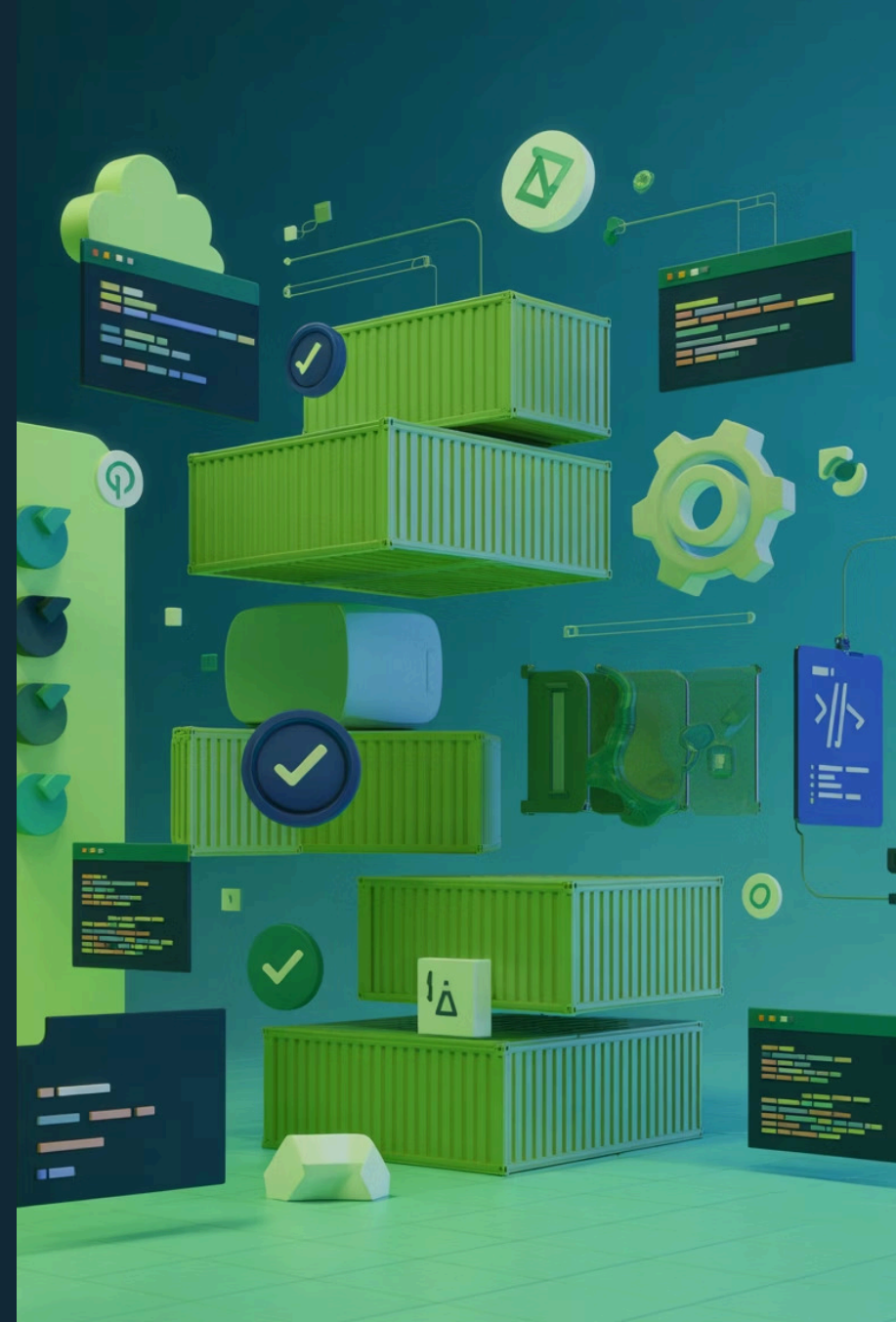


FullCycle

Curso de Testcontainers

Aprenda a usar containers em seus testes de integração para simular bancos de dados, filas e serviços reais, tornando seus testes mais confiáveis, rápidos e próximos do ambiente de produção.



Conteúdo do Curso de Testcontainers

1

1 - Introdução

Fundamentos e preparação do ambiente para testes dinâmicos com containers.

2

2 - Primeiros Passos

Criação e gerenciamento básico de containers de teste com portas e wait strategies.

3

3 - Módulos e Integrações

Integração com PostgreSQL, LocalStack (AWS), Playwright e WireMock para testes completos.

4

4 - Gerenciamento Avançado

Configuração de variáveis, redes Docker e volumes para ambientes complexos.

5

5 - Docker Compose

Orquestração de múltiplos serviços e ambientes completos com Compose.

6

6 - Otimização e Reuso

Estratégias para reutilizar containers, execução paralela e ambiente de desenvolvimento.

7

7 - Cenários Avançados

Docker-in-Docker, imagens customizadas e testes com containers especializados.

8

8 - CI/CD e Produtividade

Integração com pipelines, debugging, boas práticas e troubleshooting de testes.

A História do Testcontainers

2016: O Início

Testcontainers nasce, criado pela AtomicJar, revolucionando os testes de integração em Java com a utilização de Docker.

Expansão Multiplataforma

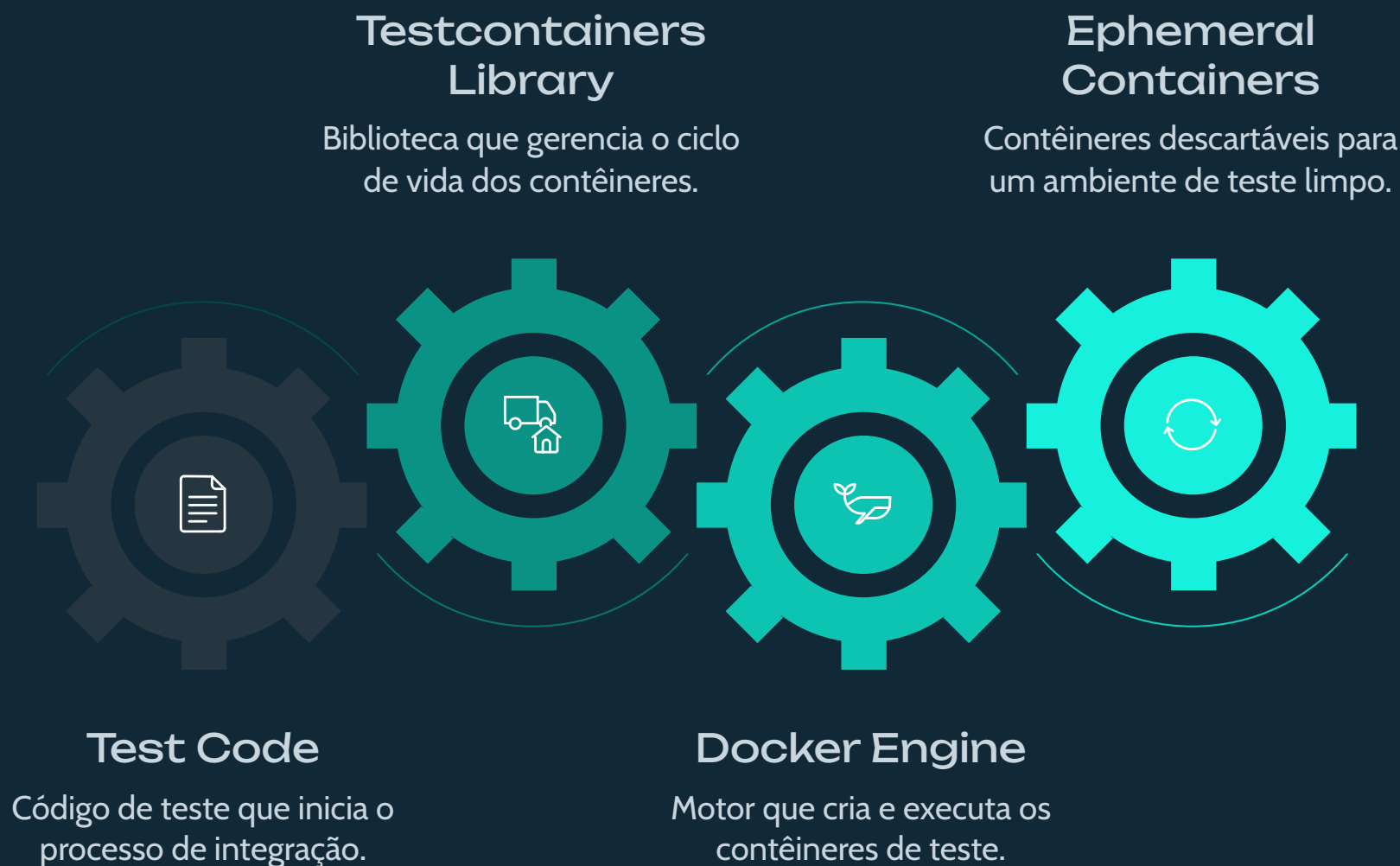
A ferramenta cresce em popularidade e expande seu suporte para outras linguagens, incluindo **Java**, **Go**, **Node.js**, **Python** e **.NET**.

2023: Docker Acquire

Testcontainers é adquirido pela Docker, fortalecendo sua posição como pilar para ambientes de desenvolvimento e testes confiáveis.



Como o Testcontainers se Comunica com o Docker



Integração com Docker Daemon

Utiliza a API do Docker via socket, funcionando perfeitamente em ambientes locais ou remotos para gerenciar containers.

Ciclo de Vida Automatizado

Gerencia o fluxo completo: baixar imagem, criar, iniciar, aguardar, executar testes, parar e remover containers automaticamente.

Gerenciamento Inteligente

Configura portas dinâmicas, aplica estratégias de espera e utiliza rótulos para um rastreamento eficiente dos containers.

Limpeza Automática

Um container auxiliar, o Ryuk, monitora e garante a remoção de todos os containers órfãos, mantendo o ambiente limpo.

💡 **Por que isso importa?** Você escreve código simples, Testcontainers lida com a complexidade, Docker executa isoladamente, e a limpeza é garantida.

Testcontainers: Linguagens e Ambientes Suportados



.NET



Golang



Java



Node.js



Python



Rust



PHP

Recentemente adicionado como
biblioteca oficial, expandindo o suporte.



✓ Bibliotecas Oficiais

Disponíveis e ativamente mantidas para **.NET**, **Go**, **Java**, **Node.js**, **Python**, **Rust** e **PHP**, garantindo estabilidade e suporte.



⚠ Bibliotecas Não Oficiais

Existem para outras linguagens, mas a variação no suporte e estabilidade pode exigir maior cautela.

Quando Usar Testcontainers



→ **Uso de dependências reais**
Execute testes contra bancos de dados, filas e serviços que são idênticos aos do ambiente de produção.

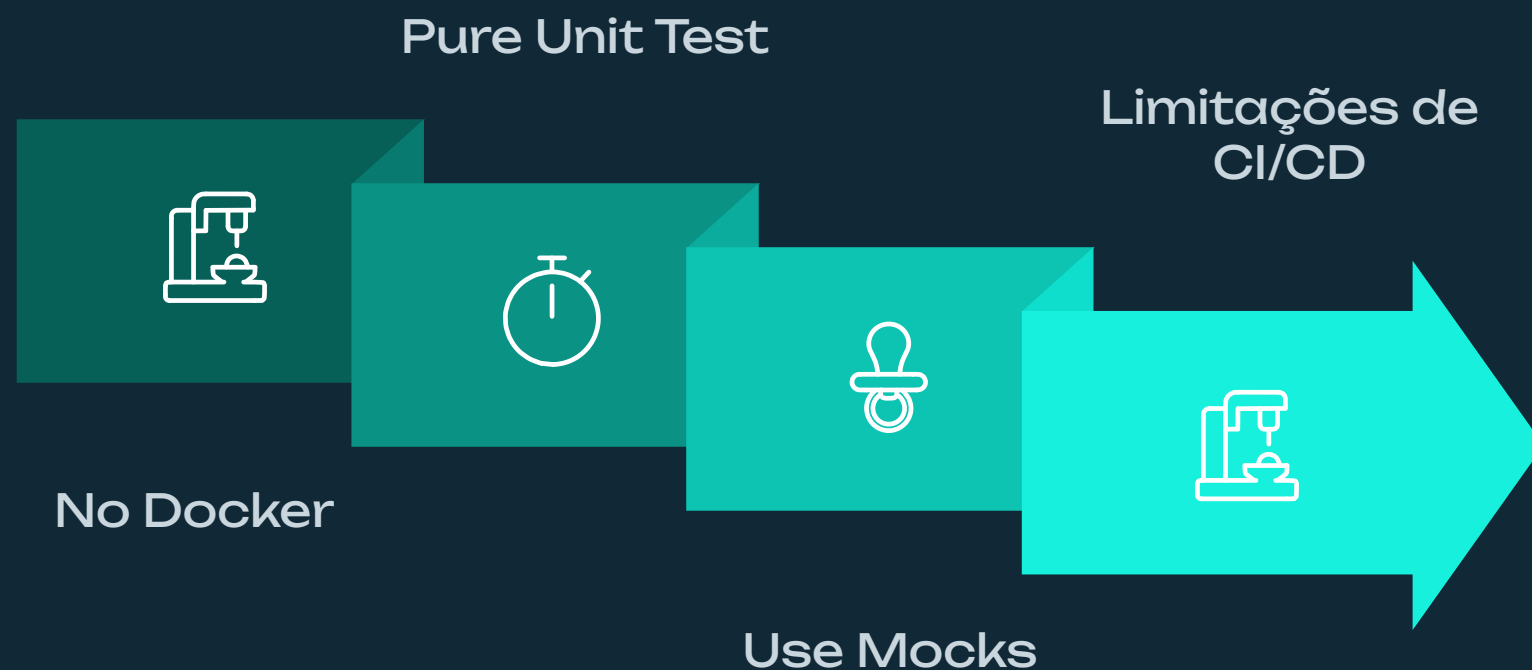
→ **Testes mais próximos da produção**
Garanta que seu código funcione corretamente no ambiente final, reduzindo surpresas na implantação.

→ **Ambientes isolados e reproduzíveis**
Cada teste é executado em um ambiente limpo e isolado, eliminando problemas de dependências compartilhadas e garantindo resultados consistentes.

→ **Feedback rápido no desenvolvimento**
Detecte e corrija problemas de integração precocemente no ciclo de desenvolvimento, acelerando a entrega.

Quando NÃO Usar Testcontainers

Testcontainers é uma ferramenta poderosa, mas entender suas limitações é crucial para um uso eficiente.



⛔ Testes Unitários Puros

Para testes que não necessitam de dependências externas e rodam em milissegundos. Mocks são mais eficientes.

🐳 Ambientes Sem Docker

Em ambientes de CI/CD ou máquinas de desenvolvimento onde o Docker não está disponível ou é restrito.

⚡ Pipelines Críticos de Performance

Containers podem adicionar sobrecarga. Avalie o custo-benefício em pipelines com restrições severas de tempo.

📋 💡 **Regra de Ouro:** Use a ferramenta certa para o trabalho certo. Testcontainers é poderoso para testes de integração realistas, mas mocks continuam essenciais para testes unitários rápidos e isolados.

Escolhendo a Ferramenta Certa

Decidir entre Mocks, Testcontainers e Testcontainers Compose depende do escopo e dos requisitos do seu teste.



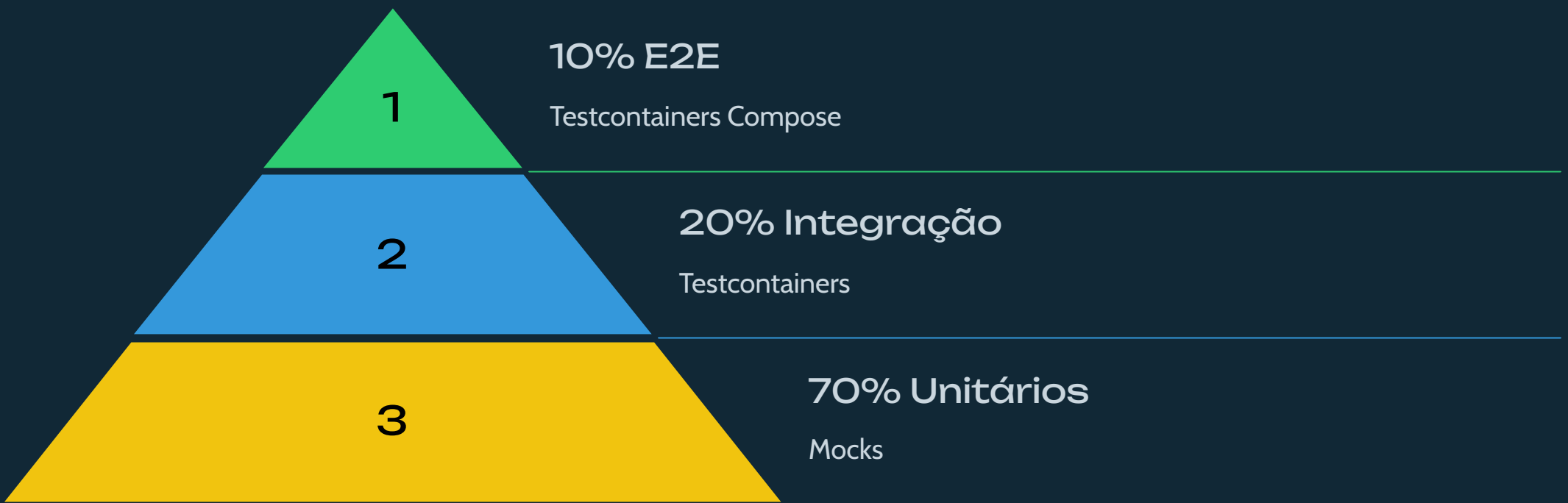
A Regra de Ouro: Use **Mocks** para **velocidade**, **Testcontainers** para **precisão**. Combine ambos para uma suíte de testes robusta e eficiente.

Aprofundando na Escolha da Ferramenta Certa

Para otimizar sua estratégia de testes, é essencial entender as nuances e cenários ideais para cada ferramenta.

	Mocks vs. Testcontainers Mocks são leves e rápidos para testes unitários, isolando a lógica de negócio. Testcontainers oferece precisão e realismo para integração, utilizando dependências reais em containers.
	Ambientes Compartilhados vs. Ephemeral Containers Ambientes compartilhados podem levar a conflitos e instabilidade. Containers efêmeros do Testcontainers garantem isolamento, reprodutibilidade e limpeza automática para cada execução de teste.
	Docker Compose Manual vs. Testcontainers Compose Configurações manuais de Docker Compose exigem orquestração e gerenciamento externos. Testcontainers Compose automatiza a inicialização, limpeza e gerenciamento de portas, simplificando testes E2E complexos.

A pirâmide de testes ilustra a proporção ideal entre os diferentes tipos de testes:



  **A Regra de Ouro:** Use **Mocks** para **velocidade**, **Testcontainers** para **precisão**. Combine ambos para uma suíte de testes robusta e eficiente.

Por que Usar Testcontainers?

Testcontainers transforma a maneira como escrevemos e executamos testes de integração, garantindo ambientes consistentes e confiáveis.



Isolamento entre testes

Zero conflitos, suítes de teste independentes e paraleláveis.



Limpeza automática

Sem containers órfãos, volumes ou redes.



Dependências versionadas

Teste múltiplas versões de DBs ou serviços com imagens Docker.



Reprodutibilidade

Mesmos resultados em Desenvolvimento, CI, Staging e Produção.



Desenvolvimento Local Simplificado

Não há necessidade de instalar serviços localmente.



Testes Confiáveis

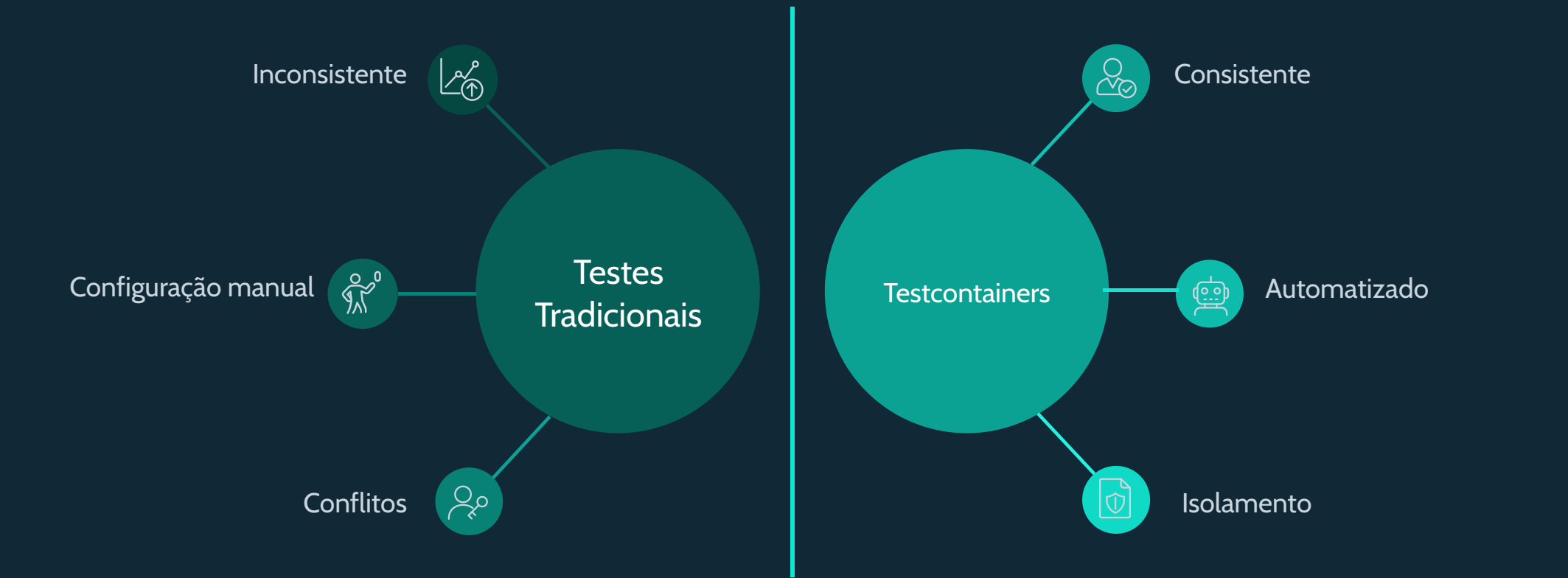
O comportamento corresponde aos sistemas de produção reais.



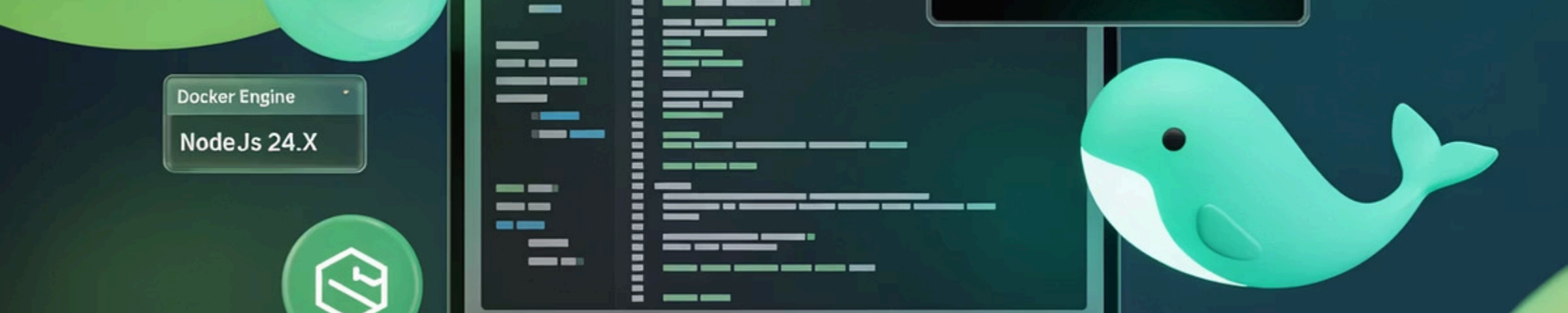
Integração Multi-serviços

Interação real entre serviços em ambientes complexos.

Testes: Tradicionais vs. Testcontainers



  **Resultado:** Testes mais próximos da produção, totalmente isolados e com manutenção manual zero.



Preparando o Ambiente de Desenvolvimento

Instale e verifique todas as dependências para começar a trabalhar com Testcontainers.

- **Docker Engine ou Docker Desktop**

Para Windows, usar o tutorial <https://github.com/codeedu/wsl2-docker-quickstart>

- **Teste a instalação do Docker**

```
docker run hello-world
```

- **Node.js**

Versão 24.x ou superior. Use **fnm** (recomendado) ou outro gerenciador de versão do NPM.

- **VS Code**

Editor de código (recomendado).